บทที่ 8

หน่วยความจำเสมือน

หน่วยความจำเสมือน (virtual memory) เป็นกลไกสำคัญของระบบปฏิบัติการที่ช่วยให้โปรแกรมสามารถทำงานได้แม้หน่วยความจำหลัก (RAM) จะมีขนาดจำกัด โดยระบบจะทำการแยกหน่วยความจำเชิงตรรกะ (logical address space) ของโปรแกรมออกจากหน่วยความจำเชิงกายภาพ (physical address space) และโหลดเฉพาะส่วนที่จำเป็นเข้าสู่หน่วยความจำในขณะนั้น ส่วนข้อมูลที่ยังไม่ได้ใช้งานจะถูกเก็บไว้ในสื่อสำรอง เช่น ฮาร์ดดิสก์ การจัดการเช่นนี้ทำให้โปรแกรมที่มีขนาดใหญ่กว่า RAM สามารถทำงานได้จริง อีกทั้งยังสนับสนุนการแชร์ข้อมูลระหว่างโพรเซส และช่วยให้การสร้างโพรเซสใหม่มีประสิทธิภาพมากขึ้น

เนื้อหาในบทนี้จะอธิบายแนวคิดหลักและกลไกที่เกี่ยวข้องกับหน่วยความจำเสมือน ตั้งแต่การจัดสรรหน้าตามคำร้องขอ (demand paging) เหตุการณ์การผิดหน้า (page fault) และการประเมินเวลาเข้าถึงหน่วยความจำจริง (effective access time) ตลอดจนกลยุทธ์การสับเปลี่ยนหน้า (page replacement algorithms) แบบต่าง ๆ เช่น FIFO, Second Chance, Clock, Optimal, NRU และ LRU รวมทั้งวิธีการจัดสรรเฟรมให้กับโพรเซสและการบริหารทรัพยากรเพื่อลดความเสี่ยงของภาวะ

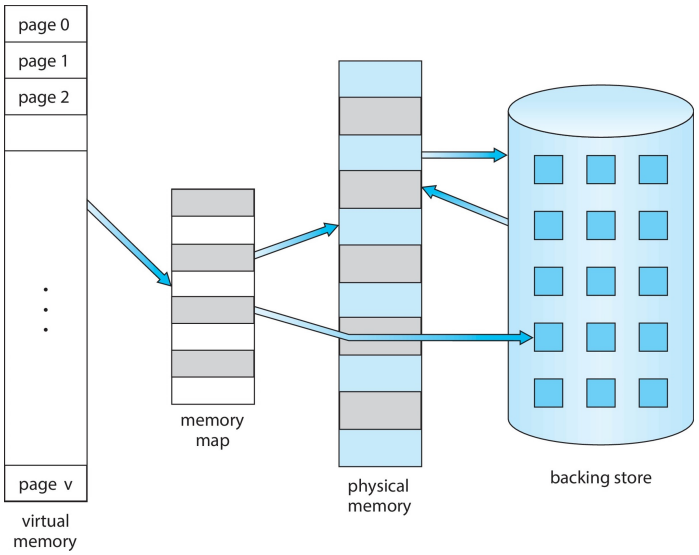
8.1 แนวคิดหน่วยความจำเสมือน

หน่วยความจำเสมือนเป็น**เทคนิคหนึ่งที่ช่วยให้โปรแกรมทำงานได้ แม้ว่าตัวโปรแกรมจะไม่ได้ถูกเก็บอยู่ในหน่วยความจำหลักทั้งหมด โดยระบบปฏิบัติการจะทำการนำเข้าเฉพาะส่วนที่จำเป็นของโปรแกรมไว้ในหน่วยความจำหลัก** ส่วนที่เหลือที่ยังไม่ได้ใช้งานจะถูกเก็บไว้ในฮาร์ดดิสก์ ตัวอย่างเช่น หากโปรแกรมต้องการพื้นที่หน่วยความจำขนาด 100 MB โปรแกรมก็ยังสามารถทำงานบนเครื่องที่มีหน่วยความจำหลัก 64 MB ได้ โดยการโหลดบางส่วนของโปรแกรม (64 MB) เข้ามาทำงานในหน่วยความจำหลัก และสลับส่วนอื่นๆ เก็บไว้ในฮาร์ดดิสก์ ซึ่งจะสลับเข้ามาเมื่อจำเป็นต้องใช้งาน ดังที่แสดงในรูปที่ 8.1

ข้อดีของวิธีนี้คือ โปรแกรมสามารถมีขนาดใหญ่กว่าหน่วยความจำหลักได้ เนื่องจากระบบจะแยกหน่วยความจำเชิงตรรกะ (logical memory) ของโปรแกรมออกจากหน่วยความจำเชิงกายภาพ (physical memory) โดยจะโหลดเฉพาะส่วนของโปรแกรมที่จำเป็นเท่านั้นเข้าไปในหน่วยความจำหลัก ทำให้ตำแหน่งหน่วยความจำเชิงตรรกะ (logical address space) สามารถมีขนาดใหญ่กว่าหน่วยความจำเชิงกายภาพ (physical address space) และยังอนุญาตให้หลายโพรเซสสามารถแชร์การใช้พื้นที่หน่วยความจำได้ ทำให้การสร้างโพรเซสเป็นไปอย่างมีประสิทธิภาพและสะดวกขึ้น

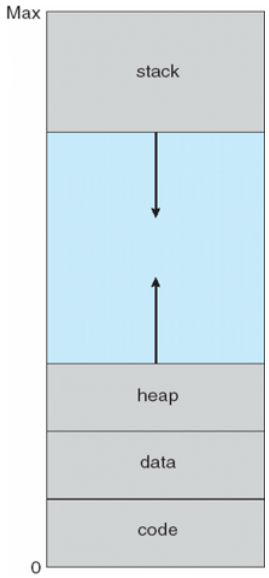
ในระบบคอมพิวเตอร์แบบ 32 บิตและ 64 บิต ตำแหน่งที่สามารถอ้างอิงได้ในส่วนของ virtual address ขนาด 32 บิต จะมีขนาดประมาณ 4 กิกะไบต์ (หรือประมาณ 4 พันล้านตำแหน่งใน 32 บิต) ส่วนขอบเขตของ physical address ขึ้นอยู่กับปริมาณหน่วยความจำที่มีอยู่ในเครื่อง สำหรับเครื่องทั่วไปอาจมีหน่วยความจำสูงสุดได้ถึง 16 กิกะไบต์ (ประมาณ 16 พันล้านตำแหน่ง) หรือเครื่องคอมพิวเตอร์ความจำต่ำๆ เช่น

256 เมกะไบต์ (ประมาณ 250 ล้านตำแหน่ง)



รูปที่ 8.1: หน่วยความจำเสมือนที่มีขนาดใหญ่กว่าหน่วยความจำหลักทางกายภาพ

พื้นที่หน่วยความจำเสมือนของโพรเซส เป็นวิธีที่แสดงให้เห็นว่าโพรเซสมองหน่วยความจำอย่างไร จากมุมมองของโพรเซส โดยหน่วยความจำเสมือนจะถูกมองเป็นที่เก็บข้อมูลที่อยู่ในลำดับตำแหน่งที่ต่อเนื่องกัน โดยโพรเซสจะเริ่มต้นที่ตำแหน่งที่อยู่เริ่มต้นที่แน่นอน คือ ตำแหน่งที่ 0 และตำแหน่งต่อๆ ไปจะเรียงกันอย่างต่อเนื่อง ดังที่แสดงในรูปที่ 8.2



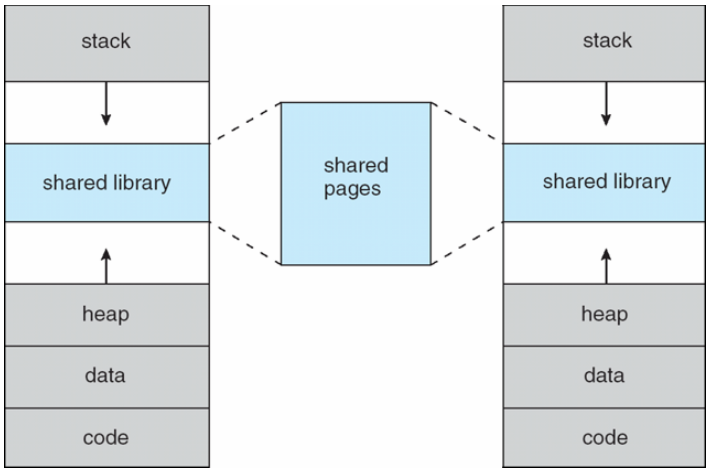
รูปที่ 8.2: พื้นที่หน่วยความจำเสมือน

การแยกหน่วยความจำเชิงตรรกะออกจากหน่วยความจำเชิงกายภาพในระบบหน่วยความจำเสมือน ช่วยให้ไฟล์ข้อมูลและหน่วยความจำสามารถถูกใช้ร่วมกันโดยโพรเซสมากกว่าหนึ่งโพรเซสได้ ซึ่งมีประโยชน์หลายประการ ดังนี้

1. **การใช้ libraries ร่วมกัน**: ระบบสามารถให้หลายโพรเซสใช้ libraries ร่วมกันได้โดยการแชร์วัตถุไปยังพื้นที่หน่วยความจำเสมือนของแต่ละโพรเซส ถึงแม้โพรเซสจะแชร์ libraries ในพื้นที่หน่วยความจำเสมือน แต่ libraries ที่อยู่ในหน่วยความจำเชิงกายภาพจะถูกแชร์ให้กับโพรเซสทั้งหมด ดังที่แสดงในรูปที่ 8.3

2. **การแชร์หน่วยความจำเสมือน**: หน่วยความจำเสมือนช่วยให้โพรเซสสามารถแชร์พื้นที่หน่วยความจำกันได้ โดยโพรเซสหนึ่งสามารถสร้างพื้นที่หน่วยความจำและแชร์กับโพรเซสอื่นๆ ได้ การแชร์นี้จะต้องพิจารณาพื้นที่ในบางส่วนของหน่วยความจำเสมือน

3. **การสร้างโพรเซสอย่างรวดเร็ว**: หน่วยความจำเสมือนช่วยให้สามารถแชร์พื้นที่ระหว่างโพรเซสที่ถูกสร้างจากฟังก์ชัน fork() และเรียกใช้งานผ่าน system call ทำให้การสร้างโพรเซสใหม่ทำได้รวดเร็วมากขึ้น



รูปที่ 8.3: แชร์ library ในหน่วยความจำเสมือน

8.2 การจัดสรรหน้าตามคำร้องขอ

การจัดสรรหน้าตามคำร้องขอ (demand paging) เป็น**วิธีการที่เพจใดๆ จะถูกนำเข้าสู่หน่วยความจำเมื่อจำเป็นต้องใช้งานเท่านั้น** เนื่องด้วยพื้นที่หน่วยความจำหลักจะมีอยู่อย่างจำกัด แนวคิดนี้เน้นว่าการทำงานของโปรแกรมหรือโพรเซสเป็นไปตามลำดับ จึงไม่จำเป็นต้องโหลดทุกเพจของโปรแกรมเข้าสู่หน่วยความจำพร้อมกันทั้งหมด

เมื่อโปรแกรมต้องการใช้งานโพรเซส ระบบจะโหลดเฉพาะเพจใด ๆ ที่จำเป็นเข้าสู่หน่วยความจำหลักโดยใช้ตัวจัดการเพจ (pager) แต่แทนที่จะโหลดทั้งหมดในคราวเดียว เราจะใช้วิธี lazy swapper ที่จะโหลดเฉพาะเพจที่ถูกเรียกใช้งานเท่านั้น ดังที่แสดงในรูปที่ 8.4 ซึ่งเพจที่ไม่ถูกใช้งานจะไม่ถูกโหลดเข้าสู่หน่วยความจำ

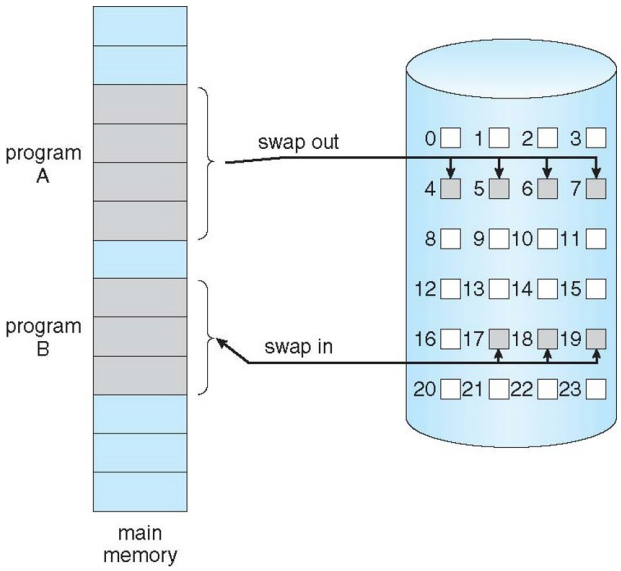
ตัวสับเปลี่ยน (swapper) มักถูกใช้ในการจัดการการสลับโพรเซส (นำเข้าและออกจากหน่วยความจำ) แต่ในกรณีของการจัดการเพจ จะใช้เพื่อโหลดเฉพาะเพจที่จำเป็นเข้าสู่หน่วยความจำ ซึ่งข้อดีของวิธีนี้คือ:

1. ลดการใช้งานอินพุต/เอาต์พุต (ลดความจำเป็นในการใช้ I/O)

2. ใช้พื้นที่หน่วยความจำน้อยลง (ประหยัดหน่วยความจำ)

3. สามารถตอบสนองได้รวดเร็วขึ้น (เพิ่มความเร็วในการตอบสนอง)

4. รองรับผู้ใช้งานได้จำนวนมากขึ้น (เพิ่มจำนวนผู้ใช้ที่ระบบสามารถรองรับได้)



รูปที่ 8.4: การจัดสรรหน้าตามคำร้องขอ

8.2.1 หลักการขั้นพื้นฐาน

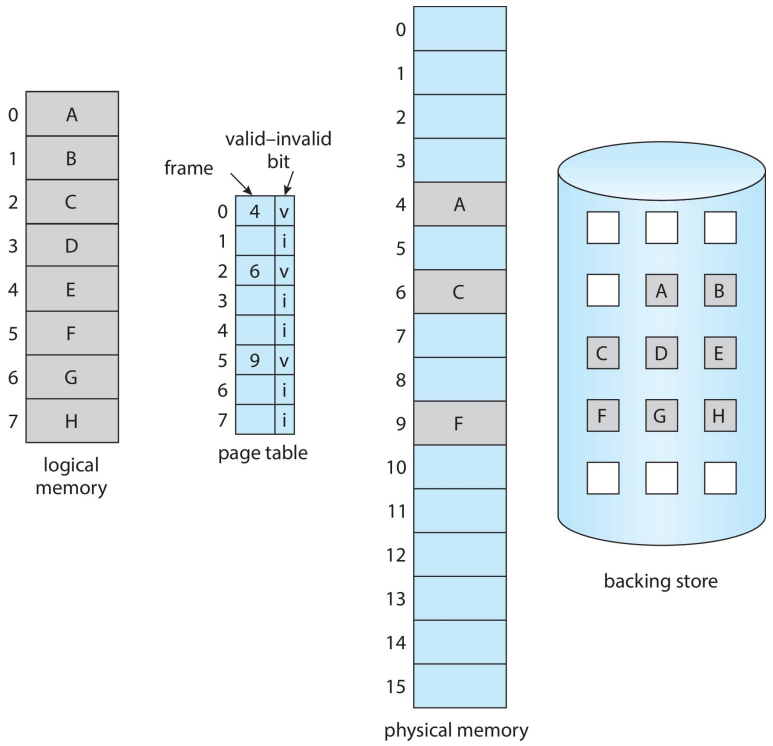
วิธีการจัดสรรหน้าตามคำร้องขอนี้ จำเป็นต้องมีการเพิ่มฮาร์ดแวร์ที่เรียกว่า บิตสถานะ (Valid-invalid bit) โดยจะเพิ่มบิตนี้ในแต่ละช่องของตารางเพจ หากบิตสถานะถูกตั้งเป็น ”valid” หมายความว่าเพจนั้นอยู่ในหน่วยความจำหลัก แต่ถ้าบิตสถานะถูกตั้งเป็น ”invalid” หมายความว่าเพจนั้นถูกเก็บไว้ในหน่วยความจำสำรอง การทำงานของตารางเพจจะเหมือนกับระบบการจัดการแบบแบ่งหน้า (paging) โดยตารางเพจจะต้องถูกนำเข้าสู่หน่วยความจำหลักพร้อมกับโพรเซส เพียงแต่ไม่จำเป็นต้องนำเพจจริงเข้ามาด้วย ดังที่แสดงในรูปที่ 8.5

การที่ระบบเรียกใช้เพจที่ไม่สามารถใช้งานได้ (ไม่ได้อยู่ในหน่วยความจำหลัก) เรียกว่า **การผิดหน้า** (page fault) เมื่อเกิดการผิดหน้าขึ้น โพรเซสที่เรียกเพจนั้นจะถูกหยุดชั่วคราว จากนั้นระบบจะดึงเพจที่ต้องการกลับมาในหน่วยความจำหลัก พร้อมทั้งอัปเดตตารางเพจจากสถานะ ใช้ไม่ได้ (invalid) เป็น ใช้ได้ (valid) หลังจากนั้นระบบจะส่งสัญญาณเพื่อให้โพรเซสดำเนินการต่ออีกครั้ง

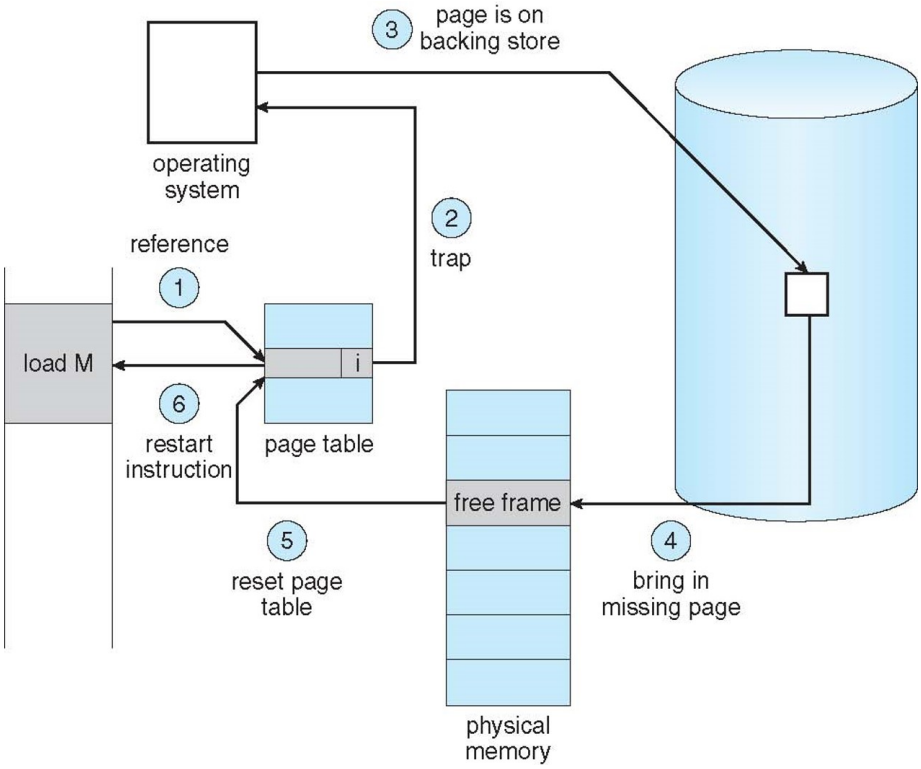
เมื่อเกิดการผิดหน้า ซึ่งหมายถึงโพรเซสพยายามใช้ส่วนของโปรแกรมที่ยังไม่ได้โหลดเข้ามาในหน่วยความจำหลักแล้ว เพจที่โพรเซสต้องการจะแสดงสถานะเป็นใช้ไม่ได้ (invalid) และระบบปฏิบัติการจะต้องจัดการปัญหานี้ตามขั้นตอน ดังที่แสดงในรูปที่ 8.6

ขั้นตอนในการจัดการการผิดหน้า (page fault) สามารถอธิบายได้ดังนี้:

1. ตรวจสอบตารางข้อมูลเฉพาะของโพรเซส (PCB) เพื่อตรวจสอบว่าหน้า (page) ที่ต้องการอ้างอิงถูกต้องหรือไม่



รูปที่ 8.5: ตารางเพจ



รูปที่ 8.6: ขั้นตอนการทำงานเมื่อเกิดการผิดหน้า

2. หากเป็นการอ้างอิงที่ผิดพลาด ให้ยกเลิกโพรเซส แต่หากการอ้างอิงถูกต้องแต่หน้าที่ต้องการไม่อยู่ในหน่วยความจำหลัก ให้ดำเนินการขั้นตอนต่อไป

3. ค้นหาเนื้อที่ว่างในหน่วยความจำจริง (frame) 1 หน้า เช่น เลือกจากรายชื่อหน้าว่าง

4. ร้องขอไปยังหน่ยความจำสำรอง (ฮาร์ดดิสก์) เพื่อนำหน้า (page) ที่ต้องการเข้ามาในเนื้อที่ว่างที่เลือกไว้

5. เมื่อหน้านั้นถูกโหลดเข้าสู่หน่วยความจำเรียบร้อยแล้ว ระบบจะอัปเดตบิตสถานะในตารางเพจเป็นใช้ได้ (valid)

6. ให้โพรเซสดำเนินการต่อจากจุดที่เกิดการผิดหน้า ซึ่งทำให้โพรเซสสามารถทำงานต่อได้เหมือนกับว่าหน้านั้นอยู่ในหน่วยความจำหลักตลอดเวลา

การใช้หน่วยความจำเสมือนช่วยให้ระบบสามารถรันโปรแกรมได้มากขึ้น โดยทำให้หน่วยความจำมีพื้นที่รวมที่มากขึ้น และสามารถทำงานหลายๆ โพรเซสพร้อมกันได้ อย่างไรก็ตาม ข้อเสียคือเมื่อมีจำนวนเพจมากขึ้น ตารางเพจก็จะใหญ่ขึ้นตามไปด้วย หากแต่ละเพจต้องใช้ 1 แถวในตารางเพจ ในกรณีที่มีถึง 1 ล้านเพจ การค้นหาแบบ linear search จะทำได้ยากและใช้เวลามากขึ้นตามไปด้วย

8.2.2 ประสิทธิภาพของการจัดสรรหน้าตามคำร้องขอ (Performance of Demand Paging)

การจัดสรรหน้าตามคำร้องขอนี้ มีผลกระทบอย่างมากต่อประสิทธิภาพของระบบ ในระบบคอมพิวเตอร์ส่วนใหญ่ เวลาที่ใช้ในการอ้างอิงหน่วยความจำเฉลี่ย (memory access time) อยู่ที่ประมาณ 10 ถึง 200 นาโนวินาที หากไม่มีการผิดหน้าเวลาการอ้างอิงเฉลี่ยจะอยู่ในช่วงนี้ แต่หากเกิดการผิดหน้า เราจะต้องอ่านเพจที่ต้องการจากฮาร์ดดิสก์ แล้วจึงทำการอ้างอิงตำแหน่งซ้ำอีกครั้ง

กำหนดให้ p เป็นความน่าจะเป็นที่จะเกิดการผิดหน้า (ค่า p มีช่วง $0 \le p \le 1$) ค่า p ควรมีค่าใกล้ 0 ให้มากที่สุด ซึ่งหมายถึงการเกิดการผิดหน้าให้น้อยที่สุด ดังนั้น สามารถคำนวณเวลาเฉลี่ยในการเข้าถึงหน่วยความจำ (Effective access time) ได้ดังนี้

$$\text{Effective access time (EAT)} = (1-p) \times ma + p \times \text{page fault time} \tag{8.1}$$

เมื่อเกิดการผิดหน้า จะมีขั้นตอนการดำเนินงานดังนี้: ระบบปฏิบัติการจะได้รับการแจ้งเตือนถึงข้อผิดพลาด โพรเซสจะต้องบันทึกค่าของรีจิสเตอร์และสถานะต่างๆ จากนั้นระบบจะตรวจสอบว่าสัญญาณที่ขัดจังหวะเป็นการผิดหน้าหรือไม่ หากเป็นการผิดหน้า ระบบจะตรวจสอบว่าหน้าที่ต้องการมีความถูกต้องและอยู่ในขอบเขตที่ได้รับอนุญาต จากนั้นจะค้นหาตำแหน่งของหน้าในหน่วยความจำสำรอง ระบบจะส่งคำร้องขอไปยังหน่วยความจำสำรองเพื่อให้อ่านหน้านั้นเข้ามายังพื้นที่ว่าง (free frame) ในระหว่างที่รอคอย คำร้องขอจะถูกจัดให้อยู่ในแถวอุปกรณ์ จานบันทึก (หน่วยความจำสำรอง) จะหมุนและค้นหาตำแหน่งที่ต้องการเพื่อโอนย้ายข้อมูลเข้าสู่พื้นที่ว่าง หลังจากนั้นจานบันทึกจะส่งสัญญาณขัดจังหวะว่าได้ดำเนินการเสร็จสิ้นแล้ว เมื่อได้รับสัญญาณนี้ ระบบจะเก็บค่าของรีจิสเตอร์และสถานะของโพรเซสอื่นที่กำลังทำงาน จากนั้นจะปรับปรุงข้อมูลในตารางเลขหน้าและตารางที่เกี่ยวข้องให้แสดงว่าหน้านั้นได้ถูกนำเข้ามาในหน่วยความจำหลักแล้ว โพรเซสจะถูกจัดให้อยู่ในแถวพร้อมเพื่อรอใช้งานหน่วยประมวลผล เมื่อถึงคราว โพรเซสจะบรรจุค่าต่างๆ ที่เก็บไว้ลงในรีจิสเตอร์และตารางเลขหน้า และทำคำสั่งที่หยุดไว้ต่อ

ขั้นตอนการจัดการการผิดหน้าแบ่งออกเป็น 3 ขั้นตอนหลักๆ คือ 1) จัดการกับสัญญาณขัดจังหวะที่เกิดจากการผิดหน้า 2) ดึงข้อมูลของหน้าที่ต้องการเข้ามาในหน่วยความจำ และ 3) ให้โพรเซสทำงานต่อจากจุดที่หยุดไป

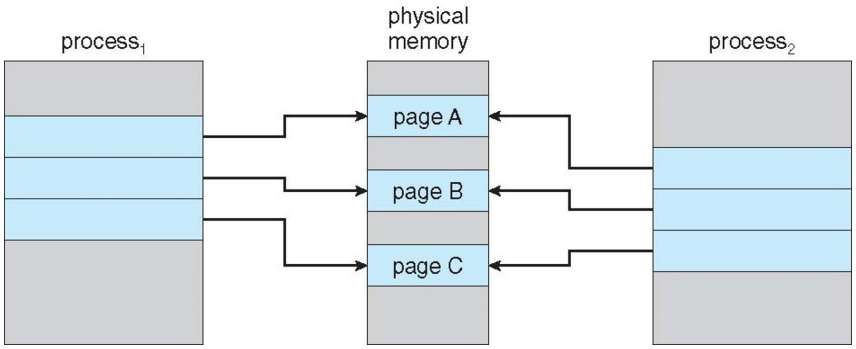
ตัวอย่างที่ 1: เวลาเฉลี่ยในการเข้าถึงหน่วยความจำ (memory access time) คือ 200 นาโนวินาที และเวลาในการจัดการการผิดหน้า (page fault time) คือ 8 มิลลิวินาที (หรือ 8,000,000 นาโนวินาที) ถ้าต้องการหาเวลาเฉลี่ยในการเข้าถึงหน่วยความจำ (effective access time) ให้อยู่ในหน่วยนาโนวินาที จะต้องคำนวณรวมทั้งสองส่วนนี้เข้าด้วยกัน โดยอาจใช้สมการในการคำนวณเพื่อให้ได้ค่าที่ถูกต้องคือ

$$\begin{aligned} EAT &= (1–p) \times 200 + p \times 8 \text{ มิลลิวินาที} \\ &= (1–p) \times 200 + p \times 8{,}000{,}000 \\ &= 200 + 7,999,800p \quad \text{นาโนวินาที} \end{aligned}$$

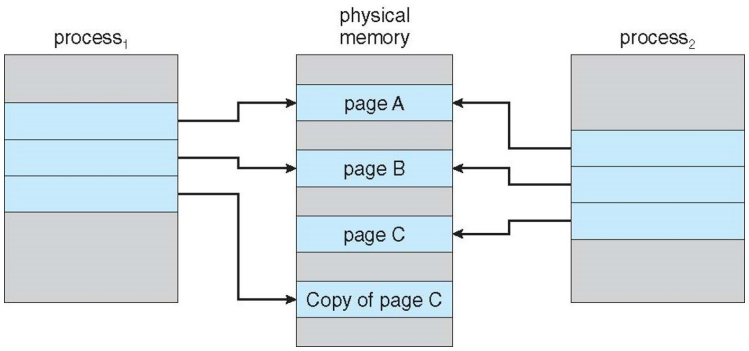
8.2.3 การสร้างโพรเซสโดยการการจัดสรรหน้าตามคำร้อง

ในหัวข้อนี้ เราจะอธิบายวิธีที่โพรเซสสามารถเริ่มต้นทำงานได้อย่างรวดเร็ว โดยเริ่มจากเพจที่มีคำสั่งแรกของโปรแกรม อย่างไรก็ตาม โพรเซสมักใช้ฟังก์ชัน fork() (ซึ่งเรียกผ่าน system call) ในการเริ่มต้นโพรเซสใหม่อย่างอ้อมๆ ระบบจะใช้วิธีการการจัดสรรหน้าตามคำร้องโดยใช้เทคนิค page sharing เทคนิคนี้ช่วยให้สามารถสร้างโพรเซสใหม่ที่รวดเร็วและขนาดเล็กขึ้น โดยจำนวนเพจใหม่ที่จำเป็นจะถูกจัดสรรให้กับโพรเซสที่ถูกสร้างขึ้น การเรียกใช้ฟังก์ชัน fork() จะสร้างโพรเซสลูก (child process) ซึ่งมีโครงสร้างเหมือนกับโพรเซสแม่ ฟังก์ชัน fork() จะคัดลอกตำแหน่งหน่วยความจำของโพรเซสแม่ให้กับโพรเซสลูก ซึ่งหมายความว่าเพจต่าง ๆ ของโพรเซสแม่จะถูกคัดลอกให้โพรเซสลูกเช่นกัน

อย่างไรก็ตาม ในกรณีที่โพรเซสลูกถูกเรียกใช้ฟังก์ชัน exec() (เรียกผ่าน system call) หลังจากการสร้าง การคัดลอกพื้นที่หน่วยความจำของโพรเซสแม่อาจไม่จำเป็น จึงใช้เทคนิคที่เรียกว่า Copy-on-Write ซึ่งทำให้โพรเซสแม่และโพรเซสลูกสามารถใช้เพจเดียวกันร่วมกันในระยะแรก (share page) โดยการเขียนลงในเพจที่ใช้ร่วมกันจะทำให้ระบบสร้างสำเนาเพจใหม่ทันที (copy-on-write) รูปที่ 8.7 และรูปที่ 8.8 แสดงการทำงานของเพจในหน่วยความจำทางกายภาพก่อนและหลังจากที่ $Process_1$ เปลี่ยนแปลง Page C



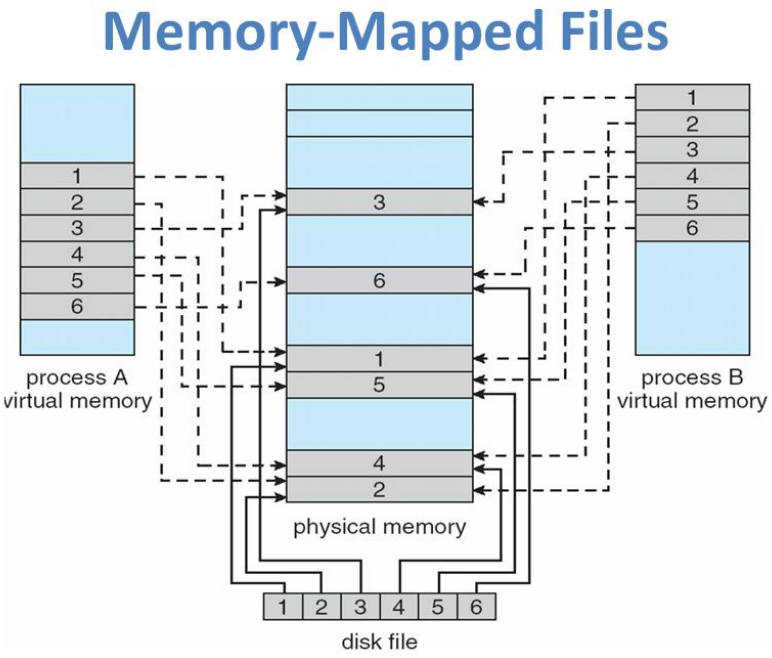
รูปที่ 8.7: ก่อนที่ $Process_1$ จะแก้ไขเพจ C



รูปที่ 8.8: หลังที่ $Process_1$ แก้ไขเพจ C

8.2.4 การเชื่อมโยงแฟ้มข้อมูลกับหน่วยความจำ

การเชื่อมโยงแฟ้มข้อมูลกับหน่วยความจำช่วยให้สามารถใช้แฟ้มข้อมูลสำหรับการอ่านและเขียนได้เหมือนกับการเข้าถึงหน่วยความจำ โดยใช้วิธีการเชื่อมโยงบล็อกของดิสก์ไปยังเพจในหน่วยความจำ โดยแฟ้มข้อมูลจะถูกกำหนดให้เริ่มอ่านตามวิธีการการจัดสรรหน้าตามคำร้อง โดยการอ่านจะเป็นการนำข้อมูลในขนาดที่เท่ากับเพจจากระบบไฟล์เข้าสู่เพจจริงของหน่วยความจำ การอ่านและเขียนแฟ้มข้อมูลจะทำงานเหมือนกับการเข้าถึงหน่วยความจำโดยตรง ทำให้การใช้งานแฟ้มข้อมูลนั้นสะดวกขึ้น โดยไม่ต้องใช้ฟังก์ชัน read() หรือ write() (ที่เรียกผ่าน system call) นอกจากนี้ยังสามารถให้หลายโพรเซสทำการเชื่อมโยงและใช้แฟ้มข้อมูลเดียวกัน โดยใช้เพจร่วมกันในหน่วยความจำได้อีกด้วย ดังแสดงในรูปที่ 8.9

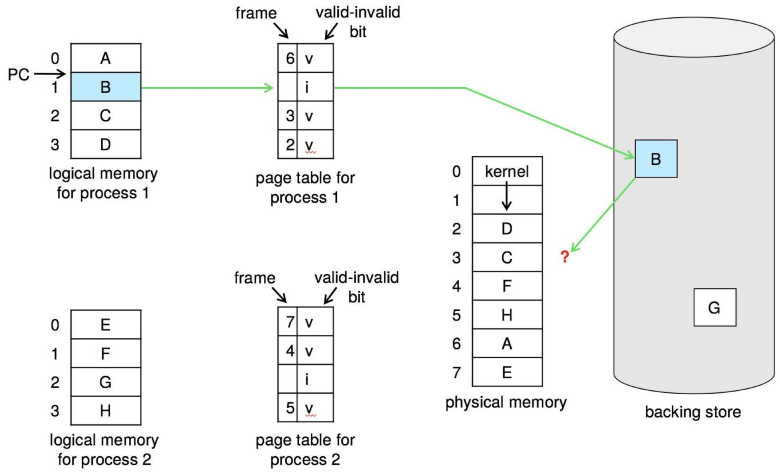


รูปที่ 8.9: การเชื่อมโยงแฟ้มข้อมูลกับหน่วยความจำ

8.3 อัลกอริทึมการสับเปลี่ยนหน้า

อัลกอริทึมการสับเปลี่ยนหน้า (page replacement) เป็นนโยบายการเลือกหน้า (เพจ) ที่อยู่ในหน่วยความจำหลักออก เพื่อนำหน้าใหม่เข้ามาแทน โดยมีปัจจัยที่ต้องพิจารณาหลายประการ เช่น

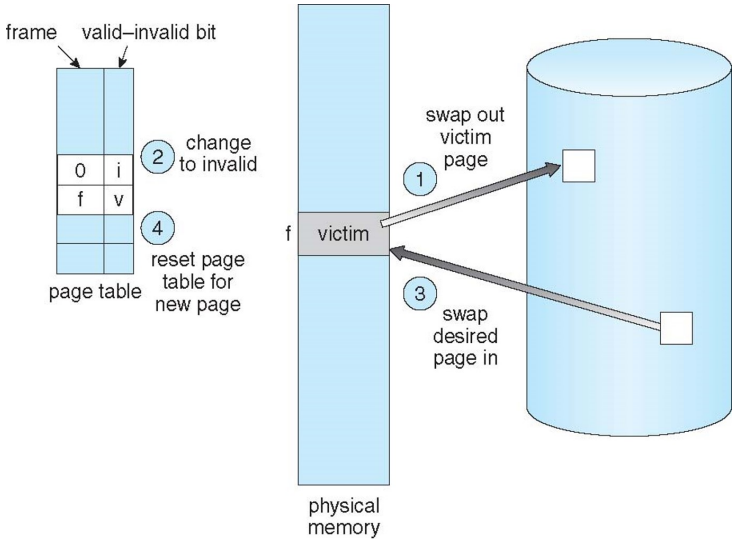
1. จำนวนเฟรมที่สามารถจัดสรรให้กับแต่ละโพรเซสได้
2. ควรจำกัดกลุ่มหน้าที่จะพิจารณาสำหรับการสับเปลี่ยนหรือไม่ เพื่อป้องกันไม่ให้โพรเซสที่ทำให้เกิดการผิดหน้าทำงาน
3. จะเลือกหน้าใดในกลุ่มหน้าที่พิจารณามาสับเปลี่ยน
4. การให้หน่วยความจำแก่โพรเซสแต่ละตัวน้อยลง จะทำให้มีโพรเซสที่เข้ามาใช้งานหน่วยความจำมากขึ้น ซึ่งจะเพิ่มโอกาสในการพบโพรเซสที่พร้อมทำงาน ลดเวลาที่เสียไปกับการสลับหน้า
5. ถ้าจำนวนหน้าของโพรเซสที่อยู่ในหน่วยความจำน้อยลง (เฟรมน้อยลง) จะทำให้อัตราการเกิดการผิดหน้าสูงขึ้น
6. การจัดสรรหน่วยความจำเพิ่มเติมให้โพรเซสหนึ่ง ๆ ไม่มีผลกระทบต่ออัตราการผิดหน้ามากนัก หากขนาดหน้าคงที่



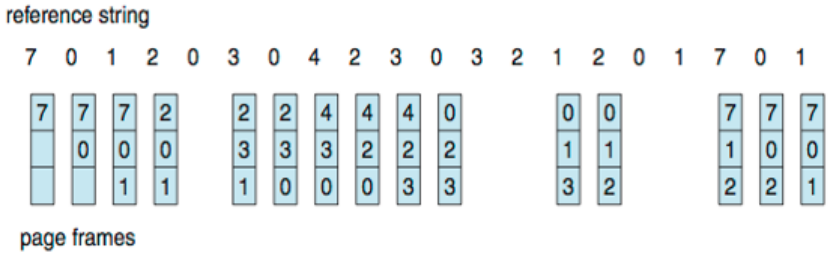
รูปที่ 8.10: ความต้องการในการสับเปลี่ยนหน้า

8.3.1 วิธีสับเปลี่ยนหน้าแบบมาก่อนออกก่อน

วิธีสับเปลี่ยนแบบมาก่อน-ออกก่อน (FIFO: First-In-First-Out algorithms) วิธีนี้เป็นวิธีที่ง่ายที่สุด โดยใช้เวลาที่หน้าแต่ละหน้าถูกนำเข้ามาในหน่วยความจำหลักเป็นเกณฑ์ในการตัดสินใจว่าจะเลือกหน้าใดออก เมื่อถึงเวลาที่ต้องสลับหน้า เราจะเลือกหน้าที่ถูกนำเข้ามานานที่สุดออกไป วิธีนี้ไม่จำเป็นต้องจดเวลาจริงที่หน้าเข้ามา เพียงแค่สร้างคิวแบบมาก่อน-ออกก่อน (FIFO queue) เพื่อเก็บหมายเลขหน้าที่อยู่ในหน่วยความจำ เมื่อมีการนำหน้าใหม่เข้ามา ก็ใส่หมายเลขหน้าใหม่ที่ท้ายแถว และเลือกหน้าเก่าที่อยู่หัวแถวออกไป



รูปที่ 8.11: การสับเปลี่ยนหน้า



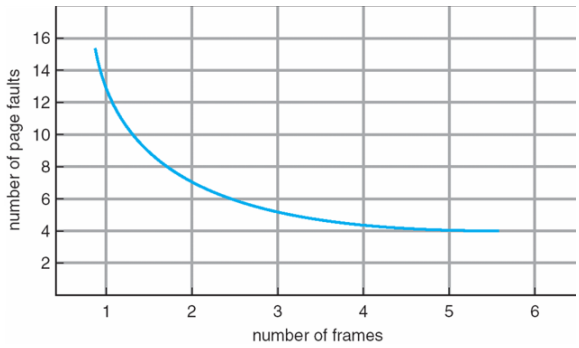
รูปที่ 8.12: First-In-First-Out (FIFO)

ในตัวอย่างที่แสดงในรูปที่ 8.12 สมมุติว่าในระบบเริ่มต้นมีแค่ 3 เฟรมและเฟรมทั้งหมดว่างอยู่ แถวหน้าที่เข้ามาในเฟรมคือ (7, 0, 1) ซึ่งจะเกิดการผิดหน้าเพราะหน้าทั้งสามยังไม่ได้ถูกโหลดเข้าเฟรมเลย ดังนั้นหน้าที่ต้องการจึงถูกโหลดเข้าสู่หน่วยความจำ หลังจากนั้นหน้า 2 จะถูกสับเปลี่ยนมาแทนหน้า 7 เพราะหน้า 7 เข้ามาก่อนเป็นหน้าแรก ส่วนครั้งถัดไปเมื่อหน้า 0 ถูกเรียกใช้งาน จะไม่เกิดการผิดหน้าเพราะหน้า 0 ยังคงอยู่ในหน่วยความจำอยู่แล้ว และไม่มีการสับเปลี่ยนเกิดขึ้น หลังจากนั้นหน้า 3 จะถูกโหลดเข้ามาแทนหน้า 0 และหน้า 0 ก็จะถูกสับเปลี่ยนกลับมาแทนหน้า 1 กระบวนการนี้จะดำเนินต่อไปเรื่อยๆ

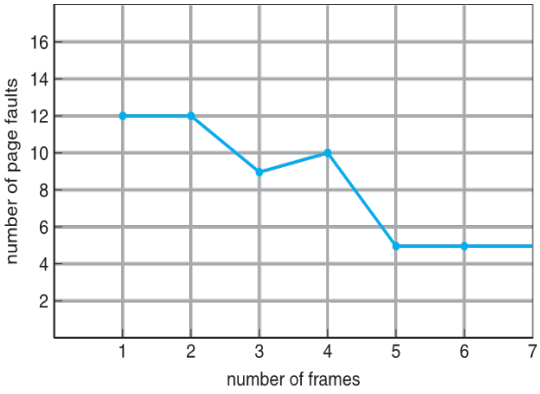
จากรูปที่ 8.12 จะเห็นได้ว่าทุกครั้งที่มีการผิดหน้า จะมีการสับเปลี่ยนหน้าใหม่เข้ามาในเฟรม ซึ่งในตัวอย่างนี้เกิดการผิดหน้าทั้งหมด 15 ครั้ง วิธีการแบบมาก่อนออกก่อน (FIFO) เป็นวิธีที่เข้าใจง่ายและสามารถเขียนโปรแกรมได้ง่าย แต่ประสิทธิภาพอาจไม่ดีนัก เนื่องจากหน้าเก่าที่ถูกสับเปลี่ยนออกไปอาจจะเป็นส่วนของโปรแกรมที่ไม่จำเป็นต้องใช้อีก หรือในบางกรณีอาจเป็นหน้าที่เก็บข้อมูลสำคัญของโปรแกรมซึ่งถูกใช้งานบ่อยครั้งตั้งแต่เริ่มต้น

สิ่งที่ควรสังเกตคือ แม้ว่าหน้าที่ยังใช้งานอยู่จะถูกสับเปลี่ยนออกไป กระบวนการทำงานของโปรแกรมยังคงดำเนินต่อไปอย่างถูกต้อง เพราะเมื่อหน้าที่ถูกใช้งานนั้นกลับมา การผิดหน้าจะเกิดขึ้นทันที และหน้าที่ไม่จำเป็นบางหน้าจะถูกสับเปลี่ยนออกแทน การสับเปลี่ยนที่ไม่ดีจะทำให้อัตราการผิดหน้าเพิ่มขึ้น และทำให้การทำงานของโปรแกรมช้าลง แต่จะไม่ทำให้ระบบทำงานผิดพลาด

เมื่อทำการทดสอบหาจำนวนครั้งที่เกิดการเปลี่ยนหน้าของหน่วยความจำ เมื่อเทียบกับจำนวนเฟรม



รูปที่ 8.13: การผิดหน้ากับจำนวนเฟรม



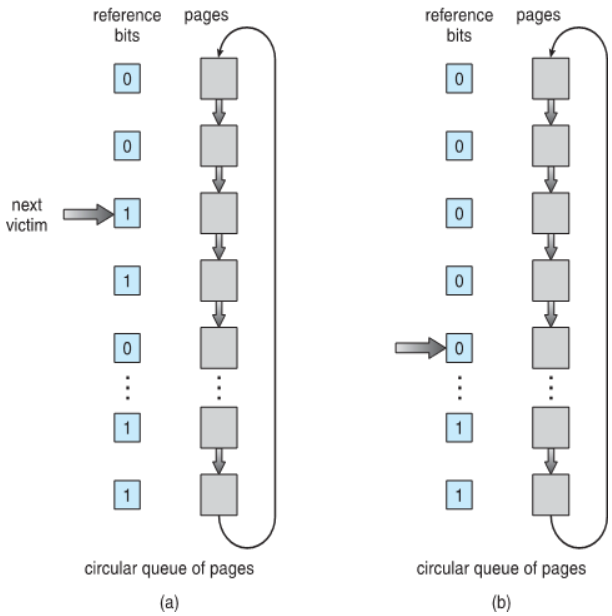
รูปที่ 8.14: Belady's Anomaly

ทั้งหมด จะได้กราฟตามภาพที่ 8.14 เราจะเห็นว่า เมื่อหน่วยความจำมี 4 เฟรม จะเกิดการเปลี่ยนหน้ามากถึง 10 ครั้ง ซึ่งมากกว่าตอนที่มีแค่ 3 เฟรม ปรากฏการณ์ที่ดูผิดปกตินี้เรียกว่า ปรากฏการณ์เบลาดี้ (Belady's anomaly) ซึ่งหมายถึงการที่บางวิธีในการจัดการเฟรมสามารถทำให้อัตราการเปลี่ยนหน้าเพิ่มขึ้นได้ แม้จะมีการเพิ่มจำนวนเฟรมก็ตาม แต่เดิมที่เราเชื่อว่าการเพิ่มหน่วยความจำจะช่วยให้กระบวนการทำงานดีขึ้นและมีประสิทธิภาพมากขึ้น แต่การค้นคว้าล่าสุดพบว่าสมมุติฐานนี้ไม่เป็นจริงเสมอไป

8.3.2 วิธีสับเปลี่ยนหน้าแบบให้โอกาสครั้งที่สอง

วิธีสับเปลี่ยนหน้าแบบให้โอกาสครั้งที่สอง (Second chance page replacement algorithms) ถูกคิดขึ้นมาเพื่อแก้ปัญหาที่เกิดจากวิธีมาก่อน-ออกก่อน โดยป้องกันไม่ให้หน้าที่ถูกใช้งานบ่อยถูกนำออกจากหน่วยความจำ วิธีการคือการตรวจสอบบิต referenced (R) ของหน้าที่อยู่มานานที่สุด ถ้าบิต R มีค่าเป็น 0 แสดงว่าหน้านั้นเก่าและไม่ได้ถูกใช้งาน ระบบสามารถสับเปลี่ยนหน้าดังกล่าวออกได้ทันที แต่ถ้าบิต R มีค่าเป็น 1 หน้านั้นจะถูกกำหนดให้บิต R เป็น 0 และถูกนำกลับไปไว้ท้ายแถวเหมือนเป็นหน้าที่เพิ่งเข้ามาใหม่ พร้อมกับการปรับเวลาให้เป็นเวลาปัจจุบัน หลังจากนั้นก็จะดำเนินการค้นหาหน้าอื่นเพื่อนำออกแทน ดังแสดงในรูปที่ 8.15

หลักการนี้คือ การค้นหาหน้าที่เก่าและไม่ได้ถูกใช้งานหรืออ้างอิง แต่ถ้าหน้าทั้งหมดในระบบถูกใช้งานอยู่ วิธีนี้จะทำงานเหมือนกับวิธีมาก่อน-ออกก่อน ยกตัวอย่างในภาพที่ 8.15 ถ้าบิต R ของทุกหน้าเป็น 1 ระบบปฏิบัติการจะทำการย้ายแต่ละหน้าไปไว้ท้ายแถวใหม่ พร้อมทั้งรีเซ็ตบิต R ให้เป็น 0 สุดท้ายหน้า A จะกลับมาเป็นหน้าที่อยู่หน้าแถวอีกครั้ง และถ้าบิต R ของหน้า A มีค่าเป็น 0 ระบบก็จะสับเปลี่ยนหน้า A ออกไป



รูปที่ 8.15: วิธีสับเปลี่ยนแบบให้โอกาสครั้งที่สอง

8.3.3 วิธีสับเปลี่ยนแบบวงรอบนาฬิกา

วิธีสับเปลี่ยนแบบวงรอบนาฬิกา (Clock page replacement algorithms) ใช้การจัดเรียงเฟรมทั้งหมดเป็นวงกลมคล้ายกับหน้าปัดนาฬิกา (ดูในรูปที่ 8.16) และมีเข็มนาฬิกาชี้ไปยังเฟรมที่เก่าที่สุด เมื่อเกิดการผิดหน้า ระบบจะตรวจสอบหน้าที่เข็มนาฬิกาชี้อยู่ ถ้าบิต R ของหน้านั้นเป็น 0 หน้านั้นจะถูกสับเปลี่ยนออกและแทนที่ด้วยหน้าใหม่ในตำแหน่งเดิม จากนั้นเข็มนาฬิกาจะเลื่อนไปยังเฟรมถัดไป 1 ตำแหน่ง แต่ถ้าบิต R เป็น 1 ระบบจะตั้งค่าใหม่ให้เป็น 0 แล้วเลื่อนเข็มไปยังหน้าถัดไป กระบวนการนี้จะทำซ้ำจนกว่าจะเจอหน้าที่มีบิต R เป็น 0 ซึ่งจะถูกสับเปลี่ยนออก วิธีนี้คล้ายกับวิธีการให้โอกาสครั้งที่สอง แต่มีความแตกต่างในวิธีการทำงานเท่านั้น

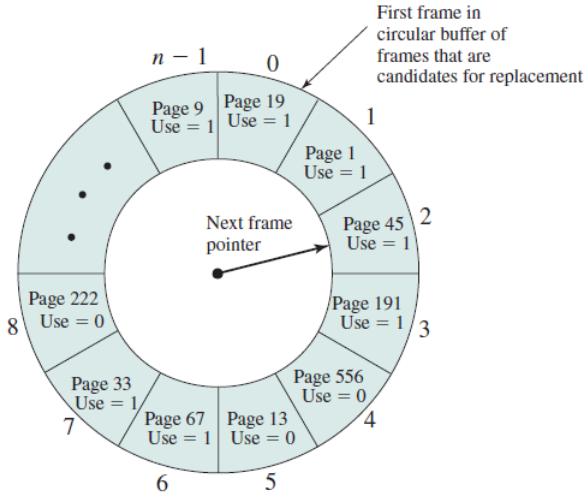
8.3.4 วิธีสับเปลี่ยนแบบที่ดีที่สุด

วิธีการสับเปลี่ยนหน้าที่ดีที่สุด (Optimal page replacement algorithms: OPT หรือ MIN) ใช้หลักการเลือกสับเปลี่ยนหน้าที่จะไม่ถูกเรียกใช้งานในอนาคตเป็นเวลานานที่สุด วิธีนี้ช่วยลดอัตราการผิดหน้าให้น้อยที่สุดสำหรับพื้นที่ที่มีจำนวนเฟรมจำกัด จากตัวอย่างในรูปที่ 8.17 แสดงให้เห็นว่าวิธี OPT นี้มีประสิทธิภาพสูงสุด ทำให้เกิดการผิดหน้าเพียง 9 ครั้ง

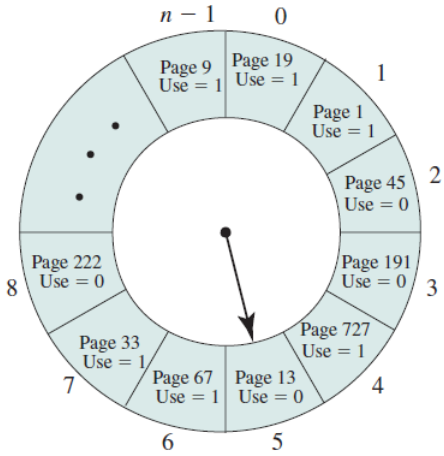
1. การเรียกใช้ 3 หน้าแรกจะทำให้เกิดการผิดหน้า 3 ครั้ง

2. การเรียกหน้า 2 จะทำให้หน้า 7 ถูกสลับออก เนื่องจากหน้า 7 จะถูกใช้อีกครั้งที่ลำดับที่ 18 ในขณะที่หน้า 0 จะถูกใช้ในลำดับที่ 5 และหน้า 1 จะถูกใช้ในลำดับที่ 14

3. การเรียกหน้า 3 จะทำให้หน้า 1 ถูกสลับออก เนื่องจากหน้า 1 เป็นหน้าที่จะถูกเรียกใช้เป็นครั้งสุดท้าย (ลำดับที่ 8 หลังจากหน้า 3) ขณะที่หน้า 0 จะถูกเรียกในลำดับที่ 1 และหน้า 2 ในลำดับที่ 3 เมื่อตรวจจากจุดที่หน้า 3 กำลังถูกใช้งาน

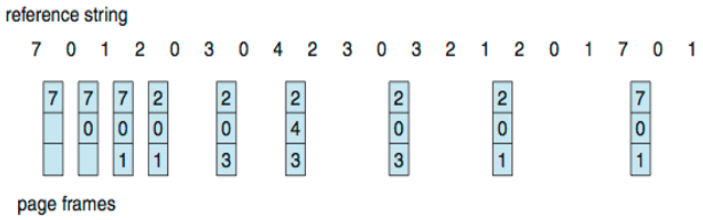


(a) State of buffer just prior to a page replacement



(b) State of buffer just after the next page replacement

รูปที่ 8.16: วิธีสับเปลี่ยนแบบวงรอบนาฬิกา

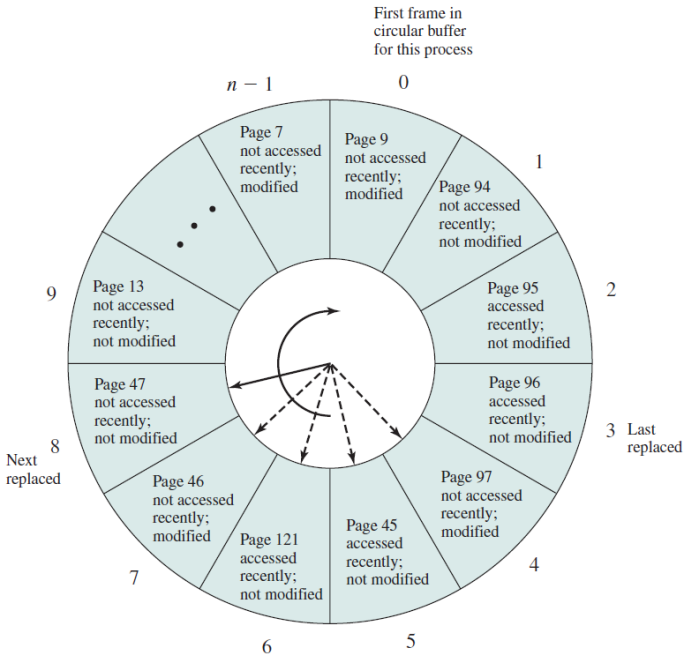


รูปที่ 8.17: Optimal Algorithm

วิธีนี้จะทำให้เกิดการผิดหน้าเพียง 9 ครั้ง ซึ่งดีกว่าวิธีมาก่อน-ออกก่อน (ที่มีการผิดหน้า 15 ครั้ง) หากไม่นับ 3 ครั้งแรกที่เกิดขึ้นอย่างแน่นอน จะเห็นได้ว่าวิธีที่ดีที่สุดนี้มีประสิทธิภาพมากกว่าวิธีมาก่อน-ออกก่อนถึง 2 เท่า อย่างไรก็ตาม วิธีนี้สร้างได้ยากในความเป็นจริง เพราะเราต้องรู้ล่วงหน้าว่าจะมีการเรียกหน้าใดและเมื่อไหร่ ดังนั้นวิธีนี้จึงใช้เพื่อการเปรียบเทียบเท่านั้น

8.3.5 วิธีสับเปลี่ยนแบบที่ไม่ได้ใช้งานออกก่อน

วิธีสับเปลี่ยนแบบที่ไม่ได้ใช้งานออกก่อน (Not recently used : NRU) ถูกพัฒนาขึ้นจากวิธีการสับเปลี่ยนแบบวงรอบนาฬิกา โดยเพิ่มการพิจารณาบิตสำคัญในตารางหน้าอีก 1 บิต เนื่องจากระบบปฏิบัติการสามารถเก็บข้อมูลสถิติได้ว่าหน้าใดถูกใช้งานหรือไม่ได้ใช้งาน โดยอิงจากบิตในตารางหน้าที่แสดงสถานะการทำงานของแต่ละหน้า บิต referenced จะถูกตั้งค่าเป็น 1 เมื่อหน้านั้นถูกเรียกใช้งาน และบิต modified จะถูกตั้งค่าเป็น 1 เมื่อมีการเปลี่ยนแปลงข้อมูลในหน้า บิตทั้งสองนี้อยู่ในแถวของตารางหน้าและจะเปลี่ยนแปลงค่าทุกครั้งที่มีการใช้งานหรือถูกอ้างอิง ดังนั้น การตั้งค่าต่างๆ จึงควรเป็นหน้าที่ของฮาร์ดแวร์ เมื่อบิตถูกตั้งค่าเป็น 1 จะคงอยู่จนกว่าระบบปฏิบัติการจะรีเซ็ตเป็น 0 โดยใช้ซอฟต์แวร์



รูปที่ 8.18: วิธีสับเปลี่ยนแบบที่ไม่ได้ใช้งานออกก่อน

บิต R และ M สามารถนำมาใช้สร้างวิธีการสับเปลี่ยนหน้าแบบใหม่ได้ โดยเมื่อโปรเซสเริ่มทำงาน ระบบปฏิบัติการจะตั้งค่าบิตทั้งสองนี้ในทุกหน้าของหน่วยความจำให้เป็น 0 และเมื่อเวลาผ่านไประยะหนึ่ง เช่น เมื่อเกิดอินเทอร์รัพต์ ระบบจะรีเซ็ตบิต R ให้เป็น 0 เพื่อแยกหน้าที่ไม่ได้ใช้งานออกจากหน้าที่กำลังถูกใช้งาน เมื่อเกิดการผิดหน้า ระบบปฏิบัติการจะตรวจสอบหน้าแต่ละหน้าและแบ่งออกเป็น 4 กลุ่มตามค่าของบิต R และ M ดังนี้:

- กลุ่มที่ 0: ไม่ถูกเรียกใช้งาน และไม่มีการเปลี่ยนแปลง
- กลุ่มที่ 1: ไม่ถูกเรียกใช้งาน แต่มีการเปลี่ยนแปลง
- กลุ่มที่ 2: ถูกเรียกใช้งาน แต่ไม่มีการเปลี่ยนแปลง
- กลุ่มที่ 3: ถูกเรียกใช้งาน และมีการเปลี่ยนแปลง

ถึงแม้บางครั้งอาจจะไม่พบหน้าใดในกลุ่มที่ 1 แต่ก็มีโอกาสเกิดขึ้นได้ ถ้าหน้าในกลุ่มที่ 3 ถูกรีเซ็ตบิต R เป็น 0 ในช่วงเวลาหนึ่ง ๆ โดยบิต M ไม่ได้ถูกรีเซ็ตด้วย เนื่องจากระบบยังต้องการทราบว่าหน้านั้นต้อง

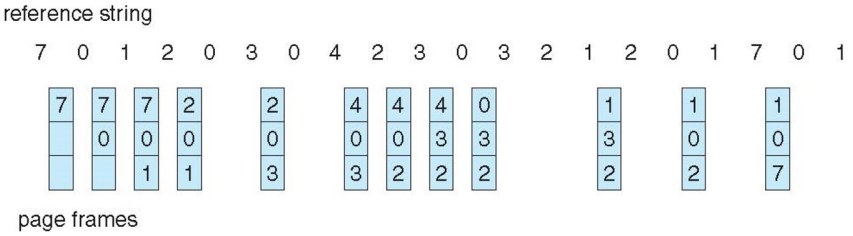
เขียนข้อมูลกลับลงดิสก์หรือไม่ ดังนั้น บิต R จะถูกเคลียร์ แต่บิต M จะยังคงเดิม ทำให้หน้านั้นย้ายไปอยู่ในกลุ่มที่ 1

วิธี NRU จะสุ่มเลือกหน้าที่จะสลับออกจากกลุ่มที่มีลำดับความสำคัญต่ำสุด (เช่น กลุ่มที่ 1 มีลำดับต่ำกว่ากลุ่มที่ 2 และ 3) โดยวิธีนี้พยายามสลับหน้าที่มีการเปลี่ยนแปลงแต่ไม่ได้ถูกใช้งานออกก่อนหน้าที่ถูกใช้งานบ่อยแต่ไม่มีการเปลี่ยนแปลง เมื่อครบวงรอบการตรวจสอบ (เช่น ทุก ๆ 20 มิลลิวินาที) ข้อดีของวิธี NRU คือความง่ายในการทำความเข้าใจและการนำไปใช้งานจริง อีกทั้งยังมีประสิทธิภาพค่อนข้างดี แม้จะไม่ดีที่สุด แต่เป็นวิธีที่ถูกใช้งานในระบบปฏิบัติการ macOS

8.3.6 การสับเปลี่ยนแบบใช้งานน้อยที่สุด-ออกก่อน

การสับเปลี่ยนแบบใช้งานน้อยที่สุด-ออกก่อน (Least recently used : LRU) วิธีนี้ใช้ข้อมูลจากอดีตเพื่อคาดการณ์การใช้งานในอนาคต โดยสลับหน้าออกที่ไม่ได้ถูกเรียกใช้งานเป็นเวลานานที่สุด เรียกว่าวิธีการใช้งานน้อยที่สุดออกก่อน วิธีนี้จะบันทึกเวลาที่แต่ละหน้าถูกใช้งานครั้งล่าสุดไว้ และเมื่อถึงเวลาที่ต้องเลือกหน้ามาสลับออก ระบบจะเลือกหน้าที่ไม่ได้ใช้งานเป็นเวลานานที่สุด (หรือหน้าที่มีเวลาน้อยที่สุด)

จะเห็นว่าวิธีนี้มองย้อนกลับไปที่การใช้งานในอดีตแทนที่จะทำนายอนาคต แม้ว่าวิธี LRU จะถือว่าได้ผลดีที่สุดในทางทฤษฎี แต่ในทางปฏิบัติกลับไม่ง่าย เพราะระบบต้องสร้างลิงก์ลิสต์ที่จัดลำดับหน้าต่างๆ ในหน่วยความจำ โดยหน้าที่ใช้งานน้อยที่สุดจะอยู่ต้นแถว และหน้าที่ใช้งานบ่อยที่สุดจะอยู่ท้ายแถว ความยากอยู่ที่ลิสต์นี้ต้องอัปเดตทุกครั้งที่มีการอ้างอิงหรือเรียกใช้งาน ซึ่งกระบวนการค้นหา ลบ หรือย้ายหน้าไปมานั้นล้วนต้องใช้เวลาการประมวลผลของหน่วยประมวลผลกลาง



รูปที่ 8.19: การสับเปลี่ยนแบบใช้งานน้อยที่สุด-ออกก่อน

8.4 การจัดสรรเฟรม

การจัดสรรเฟรม (allocation of frames) คือ กระบวนการจัดสรรหน่วยความจำที่มีอยู่ให้กับโพรเซสต่างๆ ที่กำลังทำงานในระบบคอมพิวเตอร์ โดยเฟรมเป็นส่วนหนึ่งของหน่วยความจำหลัก ซึ่งระบบปฏิบัติการจะต้องทำการจัดสรรเฟรมให้กับแต่ละโพรเซส เพื่อใช้เก็บข้อมูลและคำสั่งในการประมวลผล สำหรับการจัดสรรเฟรมนี้จะมีการกำหนดจำนวนเฟรมที่แต่ละโพรเซสได้รับอย่างเหมาะสม เพื่อให้โพรเซสสามารถทำงานได้อย่างมีประสิทธิภาพ ซึ่งการจัดสรรเฟรมมีหลายวิธี โดยวิธีที่นิยมใช้กันคือ equal allocation และ proportional allocation

8.4.1 การจัดสรรแบบเท่าเทียม (equal allocation)

เป็นวิธีการจัดสรรเฟรมให้กับโพรเซสแต่ละตัวอย่างเท่าเทียมกัน โดยไม่คำนึงถึงขนาดของโพรเซส เช่น ถ้าระบบมีเฟรม 100 เฟรม และมี 5 โพรเซส ระบบจะจัดสรรให้แต่ละโพรเซสได้รับเฟรมจำนวนเท่ากัน คือโพ

รเซสละ 20 เฟรม ซึ่งวิธีนี้เหมาะสำหรับกรณีที่แต่ละโพรเซสมีความต้องการหน่วยความจำใกล้เคียงกัน

ข้อดีของ equal allocation: ง่ายต่อการบริหารจัดการ และไม่ต้องพิจารณาขนาดหรือความซับซ้อนของโพรเซส ส่วนข้อเสียของ equal allocation: ไม่เหมาะสมในกรณีที่โพรเซสมีขนาดแตกต่างกันอย่างมาก เพราะโพรเซสขนาดใหญ่จะไม่ได้รับเฟรมเพียงพอ ในขณะที่โพรเซสขนาดเล็กอาจได้รับเฟรมเกินความจำเป็น

8.4.2 การจัดสรรตามสัดส่วน (proportional allocation)

เป็นการจัดสรรเฟรมให้กับโพรเซสแต่ละตัวตามสัดส่วนของขนาดโพรเซส โดยโพรเซสที่มีขนาดใหญ่กว่าจะได้รับเฟรมมากกว่า และโพรเซสที่มีขนาดเล็กกว่าจะได้รับเฟรมน้อยกว่า การคำนวณสัดส่วนนี้จะขึ้นอยู่กับขนาดของโพรเซสและจำนวนเฟรมที่มีอยู่ในระบบ ตัวอย่างเช่น หากระบบมี 100 เฟรม และมี 2 โพรเซส โพรเซสที่ 1 ใช้หน่วยความจำ 400 KB และโพรเซสที่ 2 ใช้หน่วยความจำ 200 KB โพรเซสที่ 1 จะได้รับเฟรม 67 เฟรม ในขณะที่โพรเซสที่ 2 จะได้รับเฟรม 33 เฟรม เนื่องจากขนาดหน่วยความจำของโพรเซสที่ 1 เป็นสองเท่าของโพรเซสที่ 2

ข้อดีของ proportional allocation: การจัดสรรเฟรมเป็นไปตามความต้องการของโพรเซส และมีประสิทธิภาพสูงในกรณีที่โพรเซสมีขนาดแตกต่างกันมาก ส่วนข้อเสียของ proportional allocation: การคำนวณอาจซับซ้อนกว่า equal allocation และไม่สามารถตอบสนองความต้องการพิเศษของโพรเซสบางโพรเซสได้ เช่น โพรเซสที่ต้องการหน่วยความจำชั่วขณะมากกว่าปกติ

8.4.3 Global Replacement และ Local Replacement

Global replacement และ local replacement เป็นวิธีการจัดการสับเปลี่ยนหน้า (page replacement) เมื่อเกิดการผิดหน้า (page fault) ในระบบคอมพิวเตอร์ที่ใช้หน่วยความจำเสมือน (virtual memory) ทั้งสองวิธีนี้มีแนวทางในการเลือกเฟรมในหน่วยความจำหลักที่จะแทนที่เมื่อมีการเรียกใช้งานหน้าจากหน่วยความจำเสมือนที่แตกต่างกัน ซึ่งรายละเอียดแต่ละวิธีมีดังนี้:

- Global replacement: ในวิธีนี้ แต่ละโพรเซสสามารถเลือกเฟรมใดๆ ก็ได้ในหน่วยความจำหลักเพื่อสับเปลี่ยนออก โดยไม่จำเป็นต้องจำกัดการเลือกเฟรมเฉพาะในโพรเซสของตัวเองเท่านั้น กล่าวอีกนัยหนึ่ง คือ โพรเซสหนึ่งสามารถเลือกสับเปลี่ยนเฟรมของโพรเซสอื่นออกได้ หากโพรเซสใดมีการเรียกหน้าที่ไม่ได้อยู่ในหน่วยความจำหลัก (เกิด page fault) ระบบจะเลือกเฟรมใดก็ได้ในหน่วยความจำหลักเพื่อทำการสับเปลี่ยน ไม่ว่าจะเป็นเฟรมของโพรเซสใดก็ตาม

- Local replacement: ในวิธีนี้ แต่ละโพรเซสสามารถเลือกสับเปลี่ยนเฟรมได้เฉพาะในขอบเขตของโพรเซสตัวเองเท่านั้น หมายความว่าเมื่อเกิด page fault ในโพรเซสหนึ่ง โพรเซสนั้นจะต้องเลือกเฟรมของตัวเองในหน่วยความจำหลักเพื่อนำไปสับเปลี่ยน โดยไม่สามารถเลือกเฟรมของโพรเซสอื่นได้

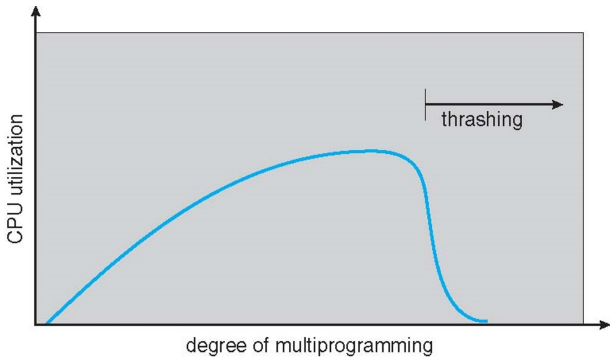
8.5 การ Thrashing

Thrashing คือภาวะที่ระบบคอมพิวเตอร์ใช้เวลาส่วนใหญ่ในการสลับหน้าเข้าและออกจากหน่วยความจำเสมือน มากกว่าที่จะประมวลผลคำสั่งจริงๆ ของโพรเซส ทำให้ระบบไม่สามารถทำงานได้อย่างมีประสิทธิภาพ การทำงานของระบบจะชะลอตัวลงอย่างมาก และในบางกรณีอาจถึงขั้นหยุดทำงานไปชั่วขณะ เพราะทุกโพรเซสพยายามเข้าถึงหน่วยความจำที่ไม่เพียงพอ

เมื่อกระบวนการใดๆ ในระบบไม่มีหน่วยความจำเพียงพอ จะทำให้เกิดอัตราการผิดหน้า (Page Fault Rate) ที่สูงขึ้น ซึ่งหน่วยความจำไม่สามารถโหลดข้อมูลที่ต้องการประมวลผลเพียงพอได้ ส่งผลให้ระบบต้อง

สลับหน้าเข้าออกจากหน่วยความจำเสมือนบ่อยครั้ง ทำให้ CPU Utilization ลดลง เนื่องจากหน่วยประมวลผลกลางต้องเสียเวลารอการเข้าถึงหน่วยความจำแทนที่จะประมวลผล เมื่อ CPU Utilization ต่ำ ระบบปฏิบัติการอาจเพิ่มระดับของ Multiprogramming เพื่อเพิ่มจำนวนโพรเซสให้ทำงานพร้อมกันในระบบมากขึ้น อย่างไรก็ตาม การเพิ่มโพรเซสเข้าไปมากขึ้นโดยไม่มีหน่วยความจำเพียงพอจะทำให้เกิดภาวะ Thrashing ซึ่งระบบจะสลับหน้าเข้าออกอย่างต่อเนื่อง โดยไม่ได้ประมวลผลจริงอย่างมีประสิทธิภาพ ดังแสดงในรูปที่ 8.20

การ Thrashing ทำให้ระบบทำงานไม่เต็มประสิทธิภาพและทำให้เวลาในการประมวลผลเพิ่มขึ้นอย่างมาก แม้จะมีโพรเซสมากมายในระบบ แต่ไม่มีโพรเซสใดที่ทำงานได้เต็มที่

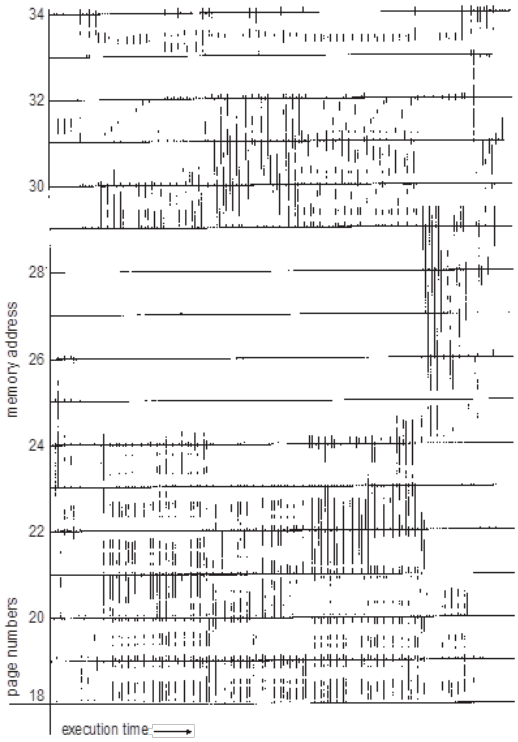


รูปที่ 8.20: Thrashing

การแก้ปัญหา Thrashing เป็นเรื่องสำคัญในการบริหารจัดการหน่วยความจำของระบบปฏิบัติการ เมื่อระบบเกิด Thrashing จะทำให้หน่วยประมวลผลกลางใช้เวลาส่วนใหญ่ในการสลับหน้าเข้าออกจากหน่วยความจำเสมือน โดยไม่มีการประมวลผลที่มีประสิทธิภาพ การแก้ปัญหานี้สามารถทำได้ด้วยวิธีการหลายวิธี ดังนี้:

1. การแก้ปัญหา Thrashing ด้วย local replacement algorithm เป็นวิธีการที่แต่ละโพรเซสจะได้รับการจัดสรรเฟรมในหน่วยความจำตามความต้องการของตัวเอง โดยไม่ส่งผลกระทบต่อโพรเซสอื่น ซึ่งแตกต่างจาก global replacement algorithm ที่จะดึงเฟรมจากโพรเซสอื่นเมื่อต้องการเฟรมเพิ่ม วิธีการนี้ช่วยลดการเกิด Thrashing ได้ เพราะแต่ละโพรเซสจะทำการสับเปลี่ยนหน้าในขอบเขตของเฟรมที่ตนเองได้รับเท่านั้น ทำให้โพรเซสหนึ่งๆ ไม่สามารถดึงเฟรมจากโพรเซสอื่นมาใช้จนทำให้เกิดการขาดแคลนเฟรมในโพรเซสอื่น ซึ่งเป็นสาเหตุหนึ่งที่ทำให้เกิด Thrashing

2. การศึกษาแต่ละโพรเซสถึงจำนวนเฟรมที่ต้องการโดยใช้ locality model ซึ่งเป็นแนวคิดที่ระบุว่าโพรเซสต่างๆ มักจะมีการใช้งานข้อมูลในหน่วยความจำที่เป็นกลุ่มๆ หรือช่วงๆ ในเวลาใดเวลาหนึ่ง ซึ่งเรียกว่า locality (ดังแสดงในรูปที่ 8.21) การศึกษาพฤติกรรมของโพรเซสในแง่ของ locality ช่วยให้ระบบสามารถจัดสรรเฟรมได้อย่างเหมาะสม โดยไม่ให้มากเกินไปหรือน้อยเกินไป จากการสังเกตว่าแต่ละโพรเซสต้องการเฟรมจำนวนเท่าใดในช่วงเวลานั้นๆ ระบบสามารถปรับจำนวนเฟรมที่จัดสรรให้กับโพรเซสตามความต้องการจริงๆ วิธีนี้ช่วยให้ลดโอกาสการเกิด Thrashing เพราะโพรเซสแต่ละตัวจะได้รับเฟรมเพียงพอต่อการประมวลผลในช่วงเวลานั้นๆ ทำให้การสลับหน้าเข้าออกน้อยลง และหน่วยประมวลผลกลางสามารถทำงานได้เต็มประสิทธิภาพ



รูปที่ 8.21: Locality

คำถามท้ายบท

1. พิจารณาลำดับการอ้างอิงหน้า (Reference String):
   7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
   หากมีจำนวนเฟรม = 3 จงคำนวณจำนวน page fault ที่เกิดขึ้นสำหรับอัลกอริทึม FIFO และทำการคำนวณซ้ำโดยใช้ LRU และ Optimal แล้วเปรียบเทียบผลลัพธ์
2. สมมติว่าระบบมีจำนวนเฟรม 4 เฟรม และลำดับการอ้างอิงคือ 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
   จงหาว่าจะเกิด page fault ทั้งหมดกี่ครั้งในกรณีที่ใช้ FIFO และ LRU